

Enhancing Vision-Language-Action Models with Intent Classification as a Pre-processing Step

Farhad Hoseyni

s.hoseyni@student.utwente.nl

Flim de Jong

f.dejong-5@student.utwente.nl

Abstract

Vision-Language-Action (VLA) models enable robots to interpret human commands but often fail with ambiguous prompts, limiting their practical use. This study introduces a two-stage approach, combining intent classification with a lightweight VLA model, to improve robustness against ambiguous prompts. We compared four classifiers (regex, Naive Bayes, RNNs, BERT) and evaluated SmolVLA on two pick-and-place tasks using non-ambiguous, slightly ambiguous, and ambiguous prompts. While BERT achieved the highest intent classification performance (F1-score: 85%), SmolVLA's task success rate declined from 75% (non-ambiguous) to 0% (ambiguous) for the marker task, with complete failure for the red block task. These findings highlight the need for improved language grounding to handle ambiguity and ensure reliable VLA model performance in real-world scenarios. A short movie for illustrative purposes can be found here: <https://www.youtube.com/watch?v=82rqNkbjcw>.

1 Introduction

With the emergence of deep neural networks in recent years, the world of human-machine interaction has evolved significantly. Natural Language Processing (NLP) plays a major role in the ability for machines to comprehend the intention of the user, but classic NLP algorithms have some limitations that make them less efficient to be used as a general-purpose agent in real-life applications. The development of deep learning algorithms and the introduction of Large Language Models (LLMs) have significantly advanced NLP tasks, enabling agentic Artificial Intelligence (AI).

Among the many capabilities of intelligent AI systems, autonomous robotic agents represent a cutting-edge area of research and development. These agents enable robots to interpret human intentions (such as commands or goals) and phys-

ically interact with their surroundings. For this purpose Vision-language-action (VLAs) models are developed as a new class of end-to-end models that aim to unify visual perception, natural language understanding, and embodied control all within a single learning framework [3]. Despite their promise, these models face notable limitations, including their limited ability to interpret ambiguous language prompts, often resulting in suboptimal task execution [5]. Some studies even suggest that VLA models largely ignore language instructions altogether [4]; therefore, to prevent the robot from making mistakes because of ambiguous prompts, the most recent models still rely on clear and structured language prompts to effectively carry out tasks [6, 1].

LeRobot [2] is an open-source robotic framework by HuggingFace that lets individuals build their own intelligent robots. One of their leading robotic arms is the SO-101 [7] which has 6 degrees of freedom and comes at an affordable price.

The LeRobot software stack is compatible with different policies including ACT [10], SmolVLA [8] and more. Additionally, the arm can be used to teleoperate a follower robot, which is useful for data gathering. As illustrated in Figure 1, the setup that is used in this research is a modified variant of the SO-101 mounted on a base called Xlerobot [9]. It features two cameras, one on the gripper and one on top of the arm, whose position is fixed.

In this research, a two-stage approach is proposed to investigate the role of intent classification to address the effects of ambiguous language prompts. During this research, it has been illustrated that in a two-stage approach, hard tasks are broken into simpler subtasks, making them easier for the VLA model to execute. Obtaining an end-to-end module to perform these tasks in a real-life application required a vast amount of data, and gathering sufficient data is a challenge; therefore, introducing the end-to-end module of VLA is out of

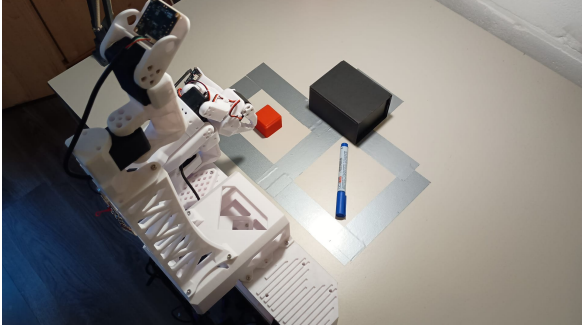


Figure 1: Image of physical testing setup

the scope of the project. The VLA model was only trained to distinguish between two different tasks to highlight its ability to respond to different intents, and to identify the limitations of a one-stage approach.

The remainder of this paper is organized into 5 sections: [section 2](#) explains about previous works, [section 3](#) focuses on the data gathering phase, [section 4](#) introduces the method that is used in this research, [section 5](#) shows the results, and [section 6](#) discusses the results and explains the conclusion of this project.

2 Related work

Vision-Language-Action (VLA) models are a relatively new area of research, with limited research exploring their robustness. In a recent study, Fei et al. [4] conducted a comprehensive robustness analysis, evaluating VLA models across seven perturbation dimensions. Notably, their findings revealed that language instructions are largely ignored by the models, highlighting a critical gap in their ability to act on natural language commands. Other research also highlight the need for clear, unambiguous prompts for successful task completion [1], [6].

3 Data

3.1 Intent Classification

The goal of intent classification data gathering is to collect enough diverse examples to represent the various ways users might interact with the agentic robot. To generate the synthetic data, four different LLMs were used, including Claude, Mistral, Deepseek, and ChatGPT, to mimic the different ways that user might express their needs. The reason to use different LLMs is to reduce the possible biases that each model might have and create a more robust dataset. The data used for classifying

the intents was obtained by prompting these LLMs, given 10 general intents or tasks that a physical agent could encounter in a tabletop environment. For each intention in each LLM, 50 data is generated as a training set, 10 for a testing set, and 5 for a test set with ambiguity. In general 2600 different prompts have been generated.

For the prompt generation, two methods of role-playing and few-shot learning is used to improve the performance of LLMs. The following prompt is used to generate the data across different LLMs:

"I want you to play the role of a data generator that generates some prompts as the input of a language model and puts them in different categories. You should keep your tone like human users that are interacting with an LLM, but you also should consider generating prompts in different variations of grammar and words that describe the task. Give me 50 representative sentences per task, as if someone were prompting the model to carry out that task. Also, give me 10 test sentences per task. Also, give me 5 more test sentences with ambiguity.

Here are some samples for prompts without ambiguity: "I want to draw a tree on a sunny afternoon." for the task of drawing a picture.

"I need to study late. Please give me some coffee." for the task of making coffee.

"My desk is a mess. Please clean it." for the task of cleaning up the desk.

The following examples are for ambiguous prompts:

"My sister lent her drawing equipment to me, so I drew a beautiful tree. Now she wants her stuff back. I think I left her stuff in the drawer." That is for opening the drawer.

"When I make coffee, I forgot to clean up my desk, and now I want to solve a puzzle, but there is no room on the desk. Make the desk tidy. for cleaning up the desk. format the output in JSON. Structure the JSON format as follows: "model": {"train": {"drawing_a_picture": {"prompts": ...}}}. The tasks are: Drawing a Picture, Cleaning Up the Desk, Folding Laundry, Sorting

Items by Color, Solving a Simple Puzzle, Making Coffee, Stacking Cubes, Putting Away Groceries, Opening the Drawer, Playing a Game.”

The intents with a small description are given in the list below.

- **Drawing a Picture:** The task involves creating visual art, such as sketches, paintings, or illustrations, based on a given prompt. The intent is to generate or replicate images, scenes, or abstract concepts using tools like pencils, paints, or digital software.
- **Cleaning Up the Desk:** This task focuses on organizing and tidying a workspace. It includes actions like removing clutter, wiping surfaces, arranging items, and disposing of trash to create a neat and functional environment.
- **Folding Laundry:** The intent is to neatly fold clothes, towels, or other fabric items after washing. The goal is to organize laundry for storage or use, often involving sorting by type or size.
- **Sorting Items by Color:** This task involves grouping objects (e.g., toys, clothes, candies, or blocks) based on their color. It emphasizes categorization and organization for aesthetic or functional purposes.
- **Solving a Simple Puzzle:** The task requires assembling or solving puzzles, such as jigsaw puzzles, Rubik’s cubes, or brain teasers. It involves logical thinking, pattern recognition, and manual dexterity.
- **Making Coffee:** This intent includes preparing coffee using various methods (e.g., drip, espresso, French press). It involves steps like grinding beans, brewing, and adding ingredients like milk or sugar.
- **Stacking Cubes:** The task involves arranging cubes or blocks to build structures, towers, or patterns. It requires spatial reasoning and fine motor skills to balance and organize the cubes.
- **Putting Away Groceries:** The intent is to unpack and store groceries in their designated places, such as the fridge, pantry, or cabinets.

It includes organizing items by category or usage.

- **Opening the Drawer:** This task involves accessing the contents of a drawer by pulling or sliding it open. It may include retrieving specific items or preparing the drawer for organization.
- **Playing a Game:** The task consists of setting up and engaging in a game, such as board games, card games, or video games. It involves following rules, taking turns, and interacting with others or a system.

3.2 Robot data

During this research, a collection of 170 episodes have been gathered by means of teleoperation. In the gathered dataset, the robotic arm carries out the task of picking up a marker and putting it in a specific place, or picking a red box and putting it in a specific place, with various amounts of prompts. The dataset can be accessed here: https://huggingface.co/datasets/Flimdejong/block_or_pen

4 Method

4.1 two-staged approach

One of the biggest challenges of using LLMs in real-time applications is hardware constraints. This issue becomes even more noticeable when integrating LLMs as components of Vision-Language-Action (VLA) models. VLA models process the input information rapidly to generate proper actions based on the current state of the robot and environment; therefore, using LLMs that could bring generalization, including working with different languages, long prompts, or ambiguous prompts in the VLA models, becomes a challenge. As illustrated in Figure 2, a two-stage approach is suggested, where the first stage uses a large model at low frequency and the second stage employs a lightweight model at high frequency. In this structure, the intent classifier processes the input prompt and environment at low frequency, breaking high-level task descriptions into simpler subtasks. Then the set of subtasks would be passed into a module called the task ordering unit, which, based on the output of the VLA model, checks if the subtask is done, and sends the next subtask to the VLA as a new prompt. The VLA model would use the simplified subtask prompt aligned with the video

frames, and states of the robot as the input tokens of a transformer and generate the set of future actions in an autoregressive manner. This process is similar to the way that a basic language model would work, which uses previous tokens to predict the next token.

In the Intent classifier, for each prompt processed by the classifier, the model outputs the intent class, which is mapped to a standardized set of prompts and passed to the VLA. To assess the benefit of intent classification on the performance of VLA models, the downstream evaluation focused on two feasible sub-tasks: "pick up the red block and put it on the black box" and "pick up the marker and put it on the black box. This serves as a representative step of the broader stacking_blocks long-horizon task. Due to time, resource, and data limitations, other intents (e.g., folding_laundry) were not evaluated downstream, with only their classification accuracy considered.

4.2 Classification models

To classify user intent, a variety of methods are selected to show the effect of different structures on the result. The classification methods include regular expressions (Regex), Naive Bayes, and small language models. The emphasis is on the best-performing model that has higher accuracy.

4.2.1 Regular Expression

Regex is a rule-based method that has a low time and space complexity. This approach required the developer to consider different ways that a certain situation might happen; therefore, it is too sensitive to different methods of expressing intent. The greedy algorithm is used for this classifier; thus, the first trigger word identifies the class of the prompt. In this research, the following words are used for different intent classifications:

- **drawing_a_picture**: draw, sketch, paint, illustrate, portrait, cartoon, watercolor, comic, drawing, painting
- **cleaning_up_the_desk**: desk, tidy, file away, office supplies, organize papers, clean up, clutter, workspace
- **folding_laundry**: fold, laundry, towels, shirts, pants, socks, sheets, sweaters, jeans, t-shirts
- **sorting_items_by_color**: (sort / arrange / separate / group) by color, lego, beads, candies, crayons, rainbow

- **solving_a_simple_puzzle**: puzzle, jigsaw, rubik, tangram, sudoku, crossword, brain teaser
- **making_coffee**: coffee, brew, espresso, cappuccino, latte, mocha, french press, percolator
- **stacking_cubes**: stack, cubes, blocks, build a tower, pyramid shape
- **putting_away_groceries**: groceries, unpack, pantry, fridge, store food, canned goods, vegetables, dairy, put away
- **opening_the_drawer**: drawer, open, pull, slide
- **playing_a_game**: game, play, chessboard, card game, video game, monopoly, poker, scrabble, jenga, checkers, bingo, uno

4.2.2 Naive Bayes

The other intent classifier that is used is naive Bayes, which is a probabilistic classification algorithm that assumes features are conditionally independent. It is efficient, easy to implement, and particularly effective for text classification tasks like spam filtering or intent classification, even with limited training data. For the intent classification task, three different variants of naive Bayes is used, including the unigram, bigram, and trigram.

4.2.3 Recurrent Neural Network

The Recurrent Neural Network (RNN) is another algorithm for processing textual inputs. RNN models process a sequence of tokens and generate a latent vector for each token. There are different variants of RNN models, including LSTM and GRU, that try to enhance the memory of the neurons. As illustrated in Table 1, in the intent classifier, a grid search has been conducted on the different hyperparameters of the model while the input text is tokenized with Google Word2Vec.

4.2.4 BERT

Currently, transformers have become the most advanced method for NLP tasks, including text classification. BERT, as one of the basic models, is a decent representative of the transformer family. For the task of intent classification, a pretrained BERT is used, while for the training, its layers are frozen and only the Multi Layer Perceptron (MLP) head is trained. This approach indicates the performance of a pretrained BERT model as a general-purpose text encoder, which is important for smart agents

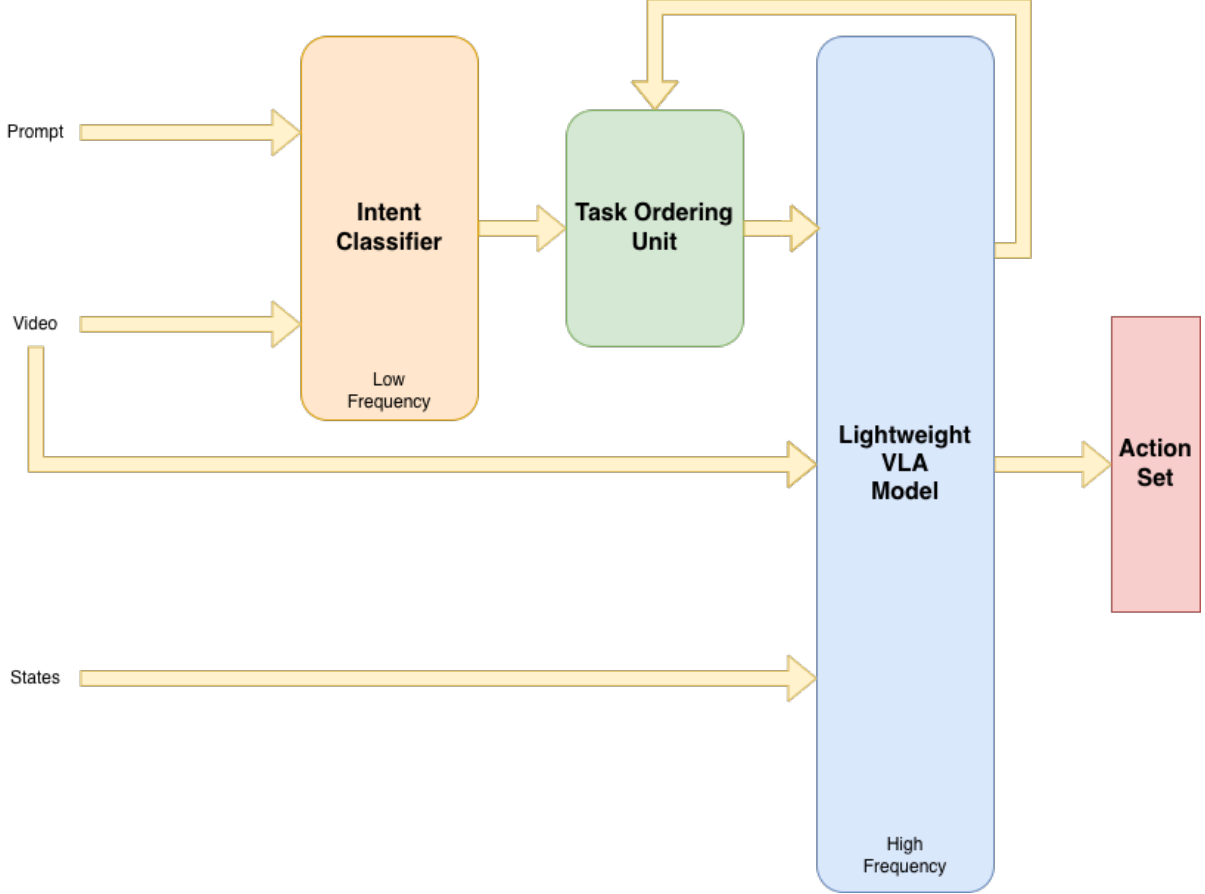


Figure 2: Two-stage approach to enhance the performance of VLA mode against ambiguity and variation in the prompt. The large VLM unit processes the high-level prompt and breaks it down to simpler tasks, and after ordering subtasks, the VLA model would do them.

RNN Type	hidden_dim	num_layers	optimizer_name	nonlinearity
RNN	16, 32, 64, 128, 256	2, 3, 4	Adam, SGD	relu, tanh
LSTM, GRU	16, 32, 64, 128, 256	2, 3, 4	Adam, SGD	–

Table 1: Hyperparameter Grid Search Space

in the two-step approach of VLA models. For the training of the MLP layer, a grid search on model width and depth has been done to select the best structure of the model.

4.3 SmolVLA

The second stage of the two-stage approach is the VLA module. SmolVLA was chosen as the VLA module, due to compatibility with the LeRobot arm, while also containing a relatively small number of parameters in comparison with other VLAs. This reduces training costs and enables deployment on low-cost hardware. SmolVLA has three variants: 0.24 billion, 0.45 billion, and 2.25 billion parameters. The SmolVLA variant that is used in this

study is the 0.45 billion variant.

As shown in Figure 3, the SmolVLA structure has two distinct parts: first, the Vision Language Model (VLM) that processes input video frames alongside the user’s prompt, and the second part is the action expert that generates a set of actions. The VLM section is a transformer-based model that uses a stack of self-attention layers to understand the intent of the user and find the corresponding objects in the environment, while the action expert uses a combination of cross-attention and self-attention layers to converge from noise to a set of reasonable actions for the robot. This model is trained in an autoregressive manner, where the output of the model depends on the previous states

of the robot, and a path of memory returns the VLM information as the input of the action expert to predict the next set of actions.

To evaluate the impact of intent classification on SmolVLA, we compare the result of non-ambiguous prompts, the baseline performance, against ambiguous prompts to assess the robustness to linguistic variation. The prompts are displayed in table 2.

Table 2: Prompts Used for SmolVLA

Task	Prompt
Marker	Baseline: Pick up the marker and place it on the black box.
	Slightly ambiguous: Please pick up the marker and position it on the black box.
	Ambiguous: Move the marker to the black box.
Red Block	Baseline: Pick up the red block and place it on the black box.
	Slightly ambiguous: Please pick up the red block and position it on the black box.
	Ambiguous: Move the red block to the black box.

4.3.1 Fine-tuning

SmolVLA is a foundation model pretrained on LeRobot community data, and contains some general "robot knowledge". But the zero-shot performance is generally not sufficient, and fine-tuning is necessary. In this research, the model is fine-tuned on 170 trajectories consisting of two tasks. The training is carried out using a script that is provided by LeRobot in PyTorch. In general, the default training hyperparameters are retained, with only the number of training steps increased to 40,000, which corresponds to ~ 8 hours of training on a RTX4090 GPU. Training with a batch size of 64 required approximately 16GB of VRAM, though peak usage could exceed this.

In earlier experiments, which involved fine-tuning the model for tasks such as picking up a red cube and touching a poker chip, significant brittleness in real-world performance was observed. Initially, it was unclear whether this came from poor data quality or other factors. However, the model’s overall success rate was exceedingly low when trained on just 50 examples. To address this issue, the dataset was expanded to 170 trajectories, explicitly incorporating spatial variation to enhance robustness.

4.3.2 Inference

The VLA model’s performance was evaluated on a physical setup with an RTX 3060 Mobile GPU, focusing on two key metrics: *task completion success* and *language grounding*.

Language grounding is defined as the initiation of the intended action (e.g., reaching for a block), regardless of whether the task is fully completed. For instance, if the model attempts to pick up a block but fails due to execution errors (e.g., dropping or improperly gripping the object), the trial is still considered a “success” in terms of language grounding. This metric evaluates the model’s ability to interpret instructions and initiate appropriate actions, rather than low-level execution precision.

Task completion success, on the other hand, refers to the absolute success rate, where a trial is only considered successful if the intended action is both initiated *and* fully completed (e.g., the block is successfully picked up and placed in the target location).

Evaluation is carried out for 10 episodes (20 for the marker task), each spanning a maximum of 60 seconds, or until the task is successfully completed.

5 Experiment and results

5.1 Intent Classification

The performance of the intent classifier as the first stage of the two-stage approach plays a critical role in the overall performance of the VLA system, mainly because every error in this stage would be considered as an input to the next stage. The [Table 3](#) shows the comparison of different models and their metrics. As expected, BERT, as a dynamic embedding model, has the best performance in most of the metrics which indicates that using a more complex method in the first stage, as the low-frequency module, can enhance the performance of the entire VLA model. In addition to the BERT model, the grid search on RNN variants show that the GRU model with the hidden layer dimension of 32, 3 3-layer stack of GRUs, and Adam optimizer would have the performance same as BERT. The reason that the GRU model has comparable results with BERT is because of the limitations of prompt dataset. In other words, if the prompts become more varied, the difference in the performance of RNN and BERT become more distinguishable. The detailed metrics for the models can be found in [Appendix A](#) and on the GitHub of the project. The

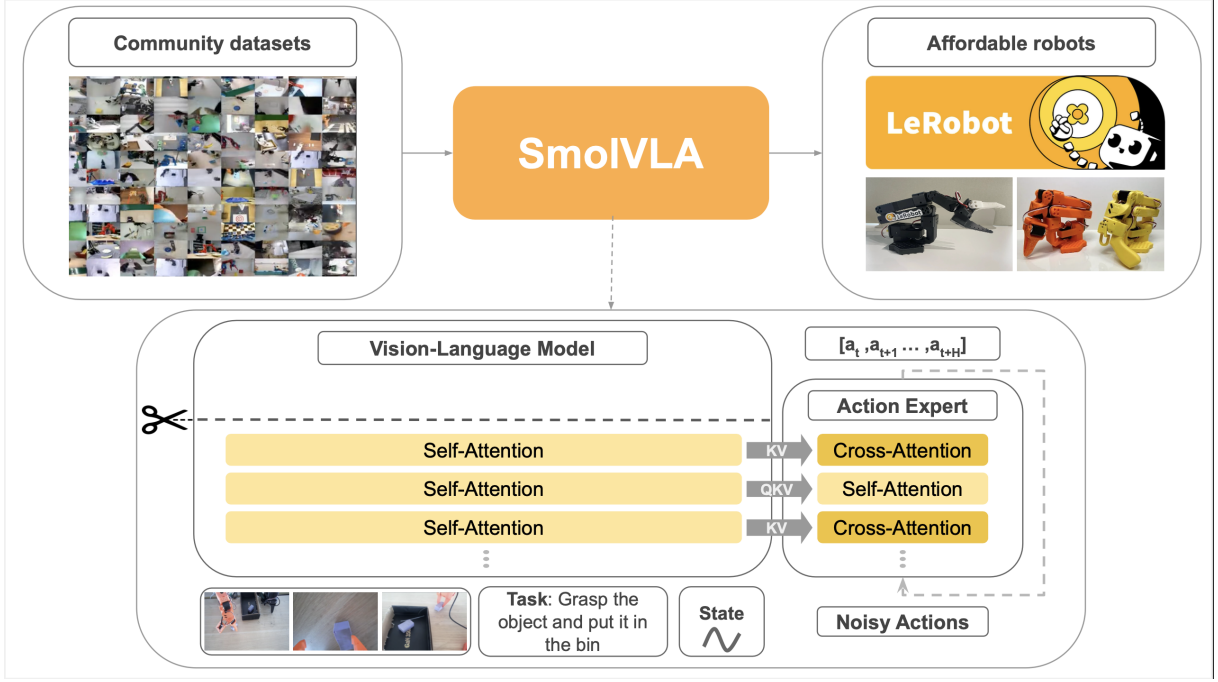


Figure 3: SmolVLA structure

details about the prompts that are classified by mistake are available in the [Appendix B](#). Some instances that are classified by mistakes show the lack of performance by the model, eg. "Fold towels and bedsheets properly," while some of the mistakes are related to bad prompting, like "Assemble by image sections," and some of the mistakes are related to ambiguity that the model could not detect such as "Yesterday I was crafting a collage for school and the papers exploded everywhere, please make it right again."

5.2 SmolVLA

[Figure 5](#) presents the training loss function of the model. The loss function’s asymptotic behavior demonstrates that the model is converging effectively during training, indicating stable training. Additionally, the gradient norms in [Figure 6](#) show a substantial decrease, indicating that the model is converging toward a local minimum.

5.2.1 Task completion success

The evaluation results for the tasks are presented in [Table 4](#). Surprisingly, the model is not able to successfully carry out the task of picking up the red block and placing it on the black box, hence the zeros in [Table 4](#). Despite showing the correct general approach, e.g. grasping it in generally the right area, it consistently has a certain offset and fails.

Performance also deteriorated with increased prompt ambiguity. However, the task with the marker showed promising results, and an additional 20 evaluations (10 for clear prompt, 10 for slightly ambiguous prompt) were conducted to improve statistical confidence. [Figure 4](#) shows the success rate of the baseline prompt against the success rate of the slightly ambiguous prompt. Out of 20 evaluations, the baseline prompt achieved a success rate of 75%, against 25% when given an slightly ambiguous prompt.

5.2.2 Language grounding

The model demonstrates *partial* language grounding, as it can distinguish between tasks such as picking up the red block or the marker. However, its performance varies significantly with highly ambiguous prompts: the marker task retains some grounding (e.g., attempting the correct action), while the block task completely fails, resulting in random behavior.

Additional Observations:

- When prompted **without any instruction**, the robot defaulted to moving toward the pen but missed it by a slight margin.
- When the **positions of the marker and block were swapped** and the original marker prompt was used, the robot moved directly to the ex-

Model	Recall	Precision	F1-score (macro)	Accuracy (micro)
Regex	0.59	0.82	0.66	0.65
NB unigram	0.79	0.79	0.79	0.86
NB bigram	0.73	0.75	0.74	0.81
NB trigram	0.59	0.79	0.64	0.65
GRU Model	0.84	0.84	0.84	0.84
BERT Model	0.85	0.85	0.85	0.84

Table 3: Comparison of Classification Model Performance Metrics

Task	Prompt 1 (Baseline)	Prompt 2 (Slightly Ambiguous)	Prompt 3 (Ambiguous)
Picking up a red block	0/10	0/10	0/10
Picking up a marker	15/20	8/20	0/10

Table 4: Performance of SmolVLA on Object Picking Tasks

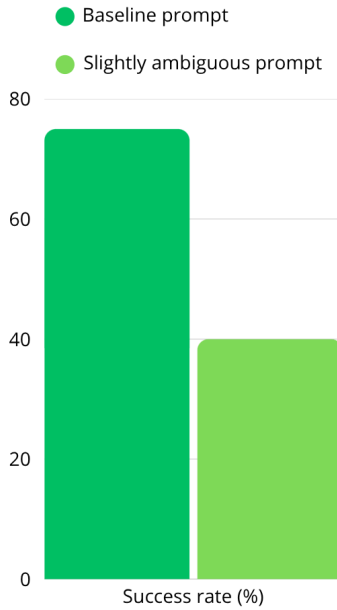


Figure 4: Task success rate (%) for picking up the marker and placing it on the black box, comparing non-ambiguous and ambiguous prompts.

pected location of the marker rather than the actual location of the object.

6 Discussion and conclusion

6.1 Discussion

One of the most pressing NLP-related challenges for language models is the ambiguity of input prompts, which directly impacts the performance of VLA models. A two-stage approach offers a promising solution: it standardizes prompts to en-

hance model performance while addressing ambiguity.

In this research, the first stage uses an intent classifier to interpret user intent and select from a set of predefined subtasks. With a slight modification, however, the intent classification block could dynamically generate subtask prompts for the next stage, rather than simply choosing from predefined options and potentially enabling long horizon behaviour. The errors in the first stage module are propagated through the entire process. Therefore obtaining a good performance by fine tuning or prompt engineering is critical. Moreover, the two stage approach introduces explainability and interpretability for the VLA model. Monitoring the intent classifier’s output clarifies the robot’s intended task, which is essential for implementing safety regulations and preventing prompts that could endanger human safety. Additionally, for the interoperability, a third party can modify the subtasks and interact with the robot on a lower level.

During the training of the intent classification unit, basic methods like naive Bayesian and GRU classifier showed remarkable performance that suggest a lightweight model can play the role of the first stage effectively. But the best performance is obtained with the BERT model with balanced performance across all of the classes. Moreover for the more complex scenarios, it is expected that other methods face a serious decrease in the performance while the transformer based models retain their capabilities. To achieve a more generalizable

model, it is essential to either adopt transformer-based generative language models or modify the architecture. For example, by incorporating reasoning traces. These approaches enable the system to better adapt to diverse user situations and requirements.

The training results for the two tasks of picking a up marker and picking up a block show acceptable performance in comparison to the size of the dataset and expected results from the SmolVLA paper. However, the out of distribution prompts reveal serious limitations regarding the model’s robustness. The ambiguity in the input prompt of the model affects the performance significantly. A two-stage approach, however, could effectively address these challenges.

Additionally, even minor variations in object placement within the environment significantly affect the model’s performance. This suggests that the vision component of the VLA model may have overfit to specific object locations, highlighting the need for a dataset with greater positional diversity. Interestingly, the model performs better on marker-related tasks despite being trained on fewer training episodes compared to box-related tasks. This discrepancy may come from the environment since the use of "the black box" shares similar characteristics with the red box. To improve the model’s robustness against such variations, a more diverse and representative dataset should be collected.

Another interesting observation during the inference phase is the reduction of performance of the model when it was used for the second time. Kim et. al. [6] suggest that this problem would happen due to tear and wear of robot parts. Additionally, the lighting is also another important factor during inference. To mitigate these challenges, training in simulation offers several advantages. It allows for the isolation of the language component from confounding variables such as lighting and camera pose, and it provides a more efficient pipeline for both inference and data collection.

6.2 Conclusion

This study investigated the impact of ambiguous language on SmolVLA’s performance and explored methods for intent classification. Four intent classifiers were compared: regular expressions, Naive Bayes, RNNs, and BERT. Among the four classifiers, BERT achieved the highest F1-score (85%), demonstrating the value of context-aware models for interpreting user intent. The performance

of SmolVLA, including the effects of ambiguous prompting, was assessed on two pick and place tasks. Each task was evaluated using three types of prompts: a baseline (non-ambiguous), a slightly ambiguous, and an ambiguous prompt. Results show that the model achieved a 75% success rate with non-ambiguous prompts for the marker task. However, performance degraded with increased ambiguity, dropping to 40% for slightly ambiguous prompts and 0% for ambiguous prompts. For the red block task, all prompts resulted in a 0% success rate. The results highlight the importance of unambiguous language for reliable VLA model performance. For future work, two key directions stand out: first, replacing the intent classification model with a generative language model, and second, leveraging a larger and more diverse dataset. Additionally, further research could prioritize improving the explainability and interpretability of the model.

References

- [1] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li-Bell, Mohith Mothukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoyang Shi, and 5 others. 2024. π_0 : A vision-language-action flow model for general robot control. *Preprint*, arXiv:2410.24164.
- [2] Remi Cadene, Simon Alibert, Alexander Soare, Quentin Gallouedec, Adil Zouitine, Steven Palma, Pepijn Kooijmans, Michel Aractingi, Mustafa Shukor, Dana Aubakirova, Martino Russi, Francesco Capuano, Caroline Pascal, Jade Choghari, Jess Moss, and Thomas Wolf. 2024. Lerobot: State-of-the-art machine learning for real-world robotics in pytorch. <https://github.com/huggingface/lerobot>.
- [3] Muhayy Ud Din, Waseem Akram, Lyes Saad Saoud, Jan Rosell, and Irfan Hussain. 2025. Vision language action models in robotic manipulation: A systematic review. *Preprint*, arXiv:2507.10672.
- [4] Senyu Fei, Siyin Wang, Junhao Shi, Zihao Dai, Jikun Cai, Pengfang Qian, Li Ji, Xinze He, Shiduo Zhang, Zhaoye Fei, Jinlan Fu, Jingjing Gong, and Xipeng Qiu. 2025. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*.
- [5] Helong Huang, Min Cen, Kai Tan, Xingyue Quan, Guowei Huang, and Hong Zhang. 2025. Graphcot-vla: A 3d spatial-aware reasoning vision-language-action model for robotic manipulation with ambiguous instructions. *Preprint*, arXiv:2508.07650.
- [6] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, Quan Vuong, Thomas Kollar, Benjamin Burchfiel, Russ Tedrake, Dorsa Sadigh, Sergey Levine, Percy Liang, and Chelsea Finn. 2024. Openvla: An open-source vision-language-action model. *Preprint*, arXiv:2406.09246.
- [7] Rob Knight, Pepijn Kooijmans, Remi Cadene, Simon Alibert, Michel Aractingi, Dana Aubakirova, Adil Zouitine, Russi Martino, Steven Palma, Caroline Pascal, and Thomas Wolf. Standard Open SO-100 & SO-101 Arms.
- [8] Mustafa Shukor, Dana Aubakirova, Francesco Capuano, Pepijn Kooijmans, Steven Palma, Adil Zouitine, Michel Aractingi, Caroline Pascal, Martino Russi, Andres Marafioti, Simon Alibert, Matthieu Cord, Thomas Wolf, and Remi Cadene. 2025. Smolvla: A vision-language-action model for affordable and efficient robotics. *Preprint*, arXiv:2506.01844.
- [9] Gaotian Wang and Zhuoyi Lu. 2025. Xlerobot: A practical low-cost household dual-arm mobile robot design for general manipulation. [url<https://github.com/Vector-Wangel/XLeRobot>](https://github.com/Vector-Wangel/XLeRobot).
- [10] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. 2023. Learning fine-grained bimanual manipulation with low-cost hardware. *Preprint*, arXiv:2304.13705.

A Appendix A

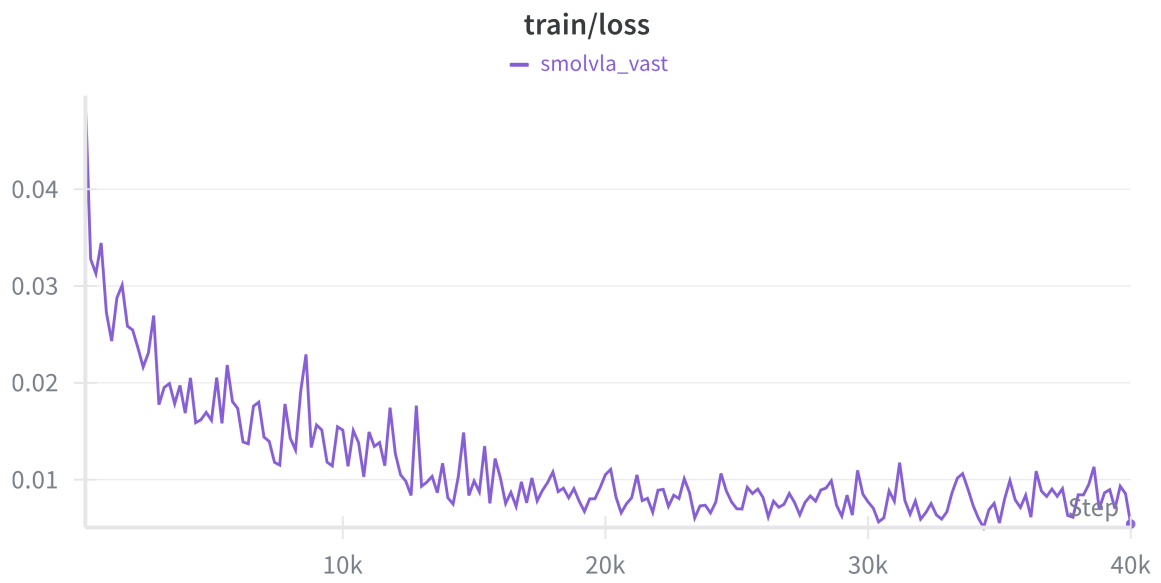


Figure 5: Training loss for SmolVLA over 40,000 steps

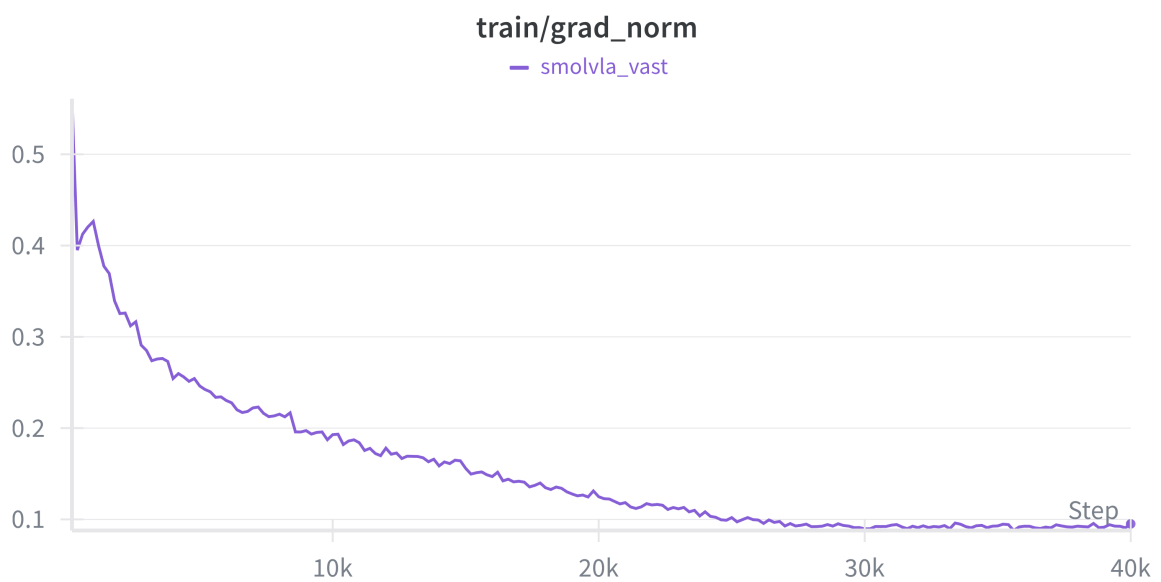


Figure 6: Gradient norms during training of SmolVLA

B Appendix B

Table 5: Analysis of Incorrect Model Predictions

Sentence	Predicted Label (ID)	Actual Label (ID)
Fold towels and bedsheets properly.	cleaning_up_the_desk (1)	folding_laundry (2)
Make sure the laundry is folded cleanly.	putting_away_groceries (7)	folding_laundry (2)
Arrange books by subject	sorting_items_by_color (3)	cleaning_up_the_desk (1)
Organize socks in pairs	putting_away_groceries (7)	folding_laundry (2)
Make piles by clothing type	sorting_items_by_color (3)	folding_laundry (2)
Organize by person’s clothes	sorting_items_by_color (3)	folding_laundry (2)
Organize drawer contents	putting_away_groceries (7)	folding_laundry (2)
Group complementary colors	drawing_a_picture (0)	sorting_items_by_color (3)
Build the center section	stacking_cubes (6)	solving_a_simple_puzzle (4)
Assemble by image sections	sorting_items_by_color (3)	solving_a_simple_puzzle (4)
Stack with color pattern	sorting_items_by_color (3)	stacking_cubes (6)
Build creative structure	putting_away_groceries (7)	stacking_cubes (6)
Access drawer contents	cleaning_up_the_desk (1)	opening_the_drawer (8)
Pull with pressure	solving_a_simple_puzzle (4)	opening_the_drawer (8)
Neatly fold the bedsheets	cleaning_up_the_desk (1)	folding_laundry (2)
Assemble this children’s puzzle for me	making_coffee (5)	solving_a_simple_puzzle (4)
Can we begin playing together?	solving_a_simple_puzzle (4)	playing_a_game (9)
I spilled some paint while sketching a map yesterday, can you handle the drawing part again?	opening_the_drawer (8)	drawing_a_picture (0)
While imagining the sunset over the hills, I couldn’t finish the art, could you complete it?	stacking_cubes (6)	drawing_a_picture (0)
There’s an unfinished page full of lines and ideas, I want it to look like an actual drawing.	sorting_items_by_color (3)	drawing_a_picture (0)
After trying to illustrate my dream last night, the paper is blank again, maybe you can fix that?	opening_the_drawer (8)	drawing_a_picture (0)
I was supposed to make a portrait of my cat but lost focus, could you take care of that picture?	opening_the_drawer (8)	drawing_a_picture (0)

Continued on next page

Sentence	Predicted Label (ID)	Actual Label (ID)
Yesterday I was crafting a collage for school and the papers exploded everywhere, please make it right again.	opening_the_drawer (8)	cleaning_up_the_desk (1)
After last night's late writing session, my working space turned chaotic, can you tidy it to normal?	opening_the_drawer (8)	cleaning_up_the_desk (1)
The desk looks like an aftermath of creativity and snacks, maybe it deserves a little attention?	playing_a_game (9)	cleaning_up_the_desk (1)
After trying on half my closet for a video call, the bed looks like a fabric storm, please resolve it.	making_coffee (5)	folding_laundry (2)
My collection of beads spilled, and now the rainbow is all mixed up, can you arrange them gracefully?	folding_laundry (2)	sorting_items_by_color (3)
Those craft buttons once had order but now look like confetti chaos, please put the hues back together.	opening_the_drawer (8)	sorting_items_by_color (3)
I tried to teach my little cousin about color families, but now the toys are confused, can you help?	putting_away_groceries (7)	sorting_items_by_color (3)
The picture puzzle from yesterday still misses logic; maybe your intellect can make sense of it?	stacking_cubes (6)	solving_a_simple_puzzle (4)
The small cardboard mystery on the table mocks me with scattered pieces, can you show it order?	playing_a_game (9)	solving_a_simple_puzzle (4)
The pieces of the ocean image seem to repel each other, perhaps your steady hand could unite them?	stacking_cubes (6)	solving_a_simple_puzzle (4)
After preparing all ingredients for my inspiration potion, I forgot the actual making, can you finish it?	playing_a_game (9)	making_coffee (5)
The aroma of caffeine is missing from my workspace; could you perform that delightful ceremony?	opening_the_drawer (8)	making_coffee (5)
The coffee grounds await a savior to turn them into something meaningful, can you perform the art?	stacking_cubes (6)	making_coffee (5)
While illustrating balance for a class, the blocks betrayed me mid-demonstration, can you fix that?	sorting_items_by_color (3)	stacking_cubes (6)
Those colorful pieces seek order after my experiment in geometry failed miserably, stack them properly.	solving_a_simple_puzzle (4)	stacking_cubes (6)
After yesterday's culinary adventure, my countertop became a grocery museum, can you restore order?	making_coffee (5)	putting_away_groceries (7)
Somewhere behind that wooden facade lies what I seek; could you help me unveil the secret within?	solving_a_simple_puzzle (4)	opening_the_drawer (8)
After organizing my art supplies, I need to create a visual representation of the sunset.	making_coffee (5)	drawing_a_picture (0)
Following the cleanup, I must illustrate the remaining clutter on my desk.	cleaning_up_the_desk (1)	drawing_a_picture (0)
Post-decluttering, I should sketch the newly arranged items for documentation.	cleaning_up_the_desk (1)	drawing_a_picture (0)
Having tidied up, I now need to depict the before-and-after state visually.	folding_laundry (2)	drawing_a_picture (0)
Subsequent to rearrangement, I must render the current setup diagrammatically.	solving_a_simple_puzzle (4)	drawing_a_picture (0)
Following the puzzle assembly, scattered pieces need systematic categorization and storage.	solving_a_simple_puzzle (4)	cleaning_up_the_desk (1)
Post-drawing activity, arrange the dispersed coloring instruments and sketchpads neatly.	sorting_items_by_color (3)	cleaning_up_the_desk (1)

Continued on next page

Sentence	Predicted Label (ID)	Actual Label (ID)
After the creative burst, restore order to the pigment-stained workspace.	solving_a_simple_puzzle (4)	cleaning_up_the_desk (1)
Subsequent to drafting, consolidate the strewn stationery and drafting tools.	folding_laundry (2)	cleaning_up_the_desk (1)
Following the spin cycle, these cloth items need tidy layering.	sorting_items_by_color (3)	folding_laundry (2)
These interlocking fragments need cognitive reassembly into their original pictorial form.	sorting_items_by_color (3)	solving_a_simple_puzzle (4)
Arrange these colored segments to reconstruct the complete visual representation.	sorting_items_by_color (3)	solving_a_simple_puzzle (4)
Methodically position these cubic objects into a towering assembly.	solving_a_simple_puzzle (4)	stacking_cubes (6)
Sequentially place these six-faced shapes into a balanced column.	solving_a_simple_puzzle (4)	stacking_cubes (6)
Logically organize these 3D squares into a graduated pile.	solving_a_simple_puzzle (4)	stacking_cubes (6)
These newly purchased provisions need categorical storage in appropriate compartments.	cleaning_up_the_desk (1)	putting_away_groceries (7)
Methodically transfer these market procurements to their assigned spatial locations.	cleaning_up_the_desk (1)	putting_away_groceries (7)
Categorically position these nutritional items into their proper repository sections.	sorting_items_by_color (3)	putting_away_groceries (7)
Execute the horizontal displacement of the storage container facade.	stacking_cubes (6)	opening_the_drawer (8)
Perform the outward motion of the bureau segregation boundary.	stacking_cubes (6)	opening_the_drawer (8)
Engage in the interactive puzzle resolution activity for entertainment.	cleaning_up_the_desk (1)	playing_a_game (9)
Commence the competitive desk organization simulation digitally.	cleaning_up_the_desk (1)	playing_a_game (9)
Initiate the virtual laundry folding procedural exercise interactively.	opening_the_drawer (8)	playing_a_game (9)
Execute the graphical coffee making sequential task virtually.	cleaning_up_the_desk (1)	playing_a_game (9)
I need to sketch the layout before organizing these shelves properly.	cleaning_up_the_desk (1)	drawing_a_picture (0)
Please illustrate how the room should look after we tidy everything.	sorting_items_by_color (3)	drawing_a_picture (0)
Map out where each item goes so I can arrange them.	sorting_items_by_color (3)	drawing_a_picture (0)
Draft the placement of these objects on paper first please.	sorting_items_by_color (3)	drawing_a_picture (0)
Outline the sequence of steps I should follow for this task.	folding_laundry (2)	drawing_a_picture (0)
I folded some shirts here earlier, now clear this workspace for me.	folding_laundry (2)	cleaning_up_the_desk (1)
The colorful markers from sorting are everywhere, please handle this area.	opening_the_drawer (8)	cleaning_up_the_desk (1)

Continued on next page

Sentence	Predicted Label (ID)	Actual Label (ID)
Categorize the groceries systematically as you store them in cabinets.	putting_away_groceries (7)	sorting_items_by_color (3)
Organize these interlocking items so they form the correct picture please.	sorting_items_by_color (3)	solving_a_simple_puzzle (4)
Stack these blocks to match the pattern shown in my drawing.	folding_laundry (2)	solving_a_simple_puzzle (4)
Tidy up by assembling these scattered fragments into their original configuration.	sorting_items_by_color (3)	solving_a_simple_puzzle (4)
Build a tower while I sketch the design for the arrangement.	solving_a_simple_puzzle (4)	stacking_cubes (6)
Arrange these blocks by color into a vertical formation please.	solving_a_simple_puzzle (4)	stacking_cubes (6)
After cleaning, assemble these geometric shapes into a neat pile structure.	folding_laundry (2)	stacking_cubes (6)
Organize these cubic objects like solving a three dimensional puzzle challenge.	solving_a_simple_puzzle (4)	stacking_cubes (6)
Fold the box template then construct a tower from these pieces.	solving_a_simple_puzzle (4)	stacking_cubes (6)
Store these food items after sorting them by their packaging color.	sorting_items_by_color (3)	putting_away_groceries (7)
Organize the kitchen supplies I bought into the appropriate storage compartments.	cleaning_up_the_desk (1)	putting_away_groceries (7)
Clean the counter then arrange these products in their designated locations.	folding_laundry (2)	putting_away_groceries (7)
Open the drawer and place these grocery items in proper positions.	folding_laundry (2)	putting_away_groceries (7)
Access the storage compartment to get supplies for my art project.	cleaning_up_the_desk (1)	opening_the_drawer (8)
I need the cabinet pulled out so I can put laundry inside.	folding_laundry (2)	opening_the_drawer (8)
Pull the furniture section to organize the colorful items inside properly.	folding_laundry (2)	opening_the_drawer (8)
Let's solve this entertaining puzzle together using the colorful pieces available.	solving_a_simple_puzzle (4)	playing_a_game (9)
Start the interactive challenge while I clean up the surrounding area.	making_coffee (5)	playing_a_game (9)
Engage with the recreational activity involving stacking and strategic placement rules.	cleaning_up_the_desk (1)	playing_a_game (9)
After making coffee, begin the competitive tabletop challenge with me now.	solving_a_simple_puzzle (4)	playing_a_game (9)
Sort the clothes by color and arrange them in the closet.	folding_laundry (2)	sorting_items_by_color (3)
Work on the puzzle and finish it before dinner.	making_coffee (5)	solving_a_simple_puzzle (4)